

25-Regex (Python-focused)

Introduction

- This lecture covers Regular Expressions (Regex), concluding the core Python segment and preparing students for data science applications.
- The focus is on introducing regex, its syntax, functions in Python, and usage for data cleaning, extraction, and validation.

Regex Fundamentals

- Regular expressions are patterns used for searching and matching parts of strings/texts by describing a pattern.
- Use-cases include extracting emails, validating credit card numbers, and dates.
- Regex is important for data science, especially in NLP and data cleaning.

Regex Syntax and Patterns

- Common symbols:
 - `.` matches any character except newline.
 - `^` matches patterns at string start; `$` matches at the end.
 - `[]` defines character sets; `[^]` negates.
 - `*` (zero/more), `+` (one/more), `?` (zero/one), `{n}` (exact n), `{n,m}`

(between n and m occurrences).

- `|` for 'or' alternatives.
- Escapes: `\d` (digit), `\D` (non-digit), `\w` (alphanumeric/underscore), `\W` (non-alphanumeric), `\s` (whitespace), `\S` (non-whitespace), `\b` (word boundary), etc.
- Parentheses `()` group patterns for matching sequences.

Regex in Python

- While the regex concept is similar across languages, syntax may slightly differ.
- Python uses the `re` module; always use raw strings (prefix `r`) to avoid escape sequence issues.

Regex Methods

- `match()`: checks for pattern only at the string's start, returns match object or None.
- `search()`: finds the first occurrence of a pattern anywhere, returns match object or None.
- `findall()`: finds all occurrences and returns a list of matches.
- `finditer()`: returns an iterator of match objects, allowing extraction of groups, spans, etc.

Pattern Matching Examples

- Matching emails: Defining a regex to match standard email patterns with various domain names.

- Validating formats: Demonstrations of pattern extraction (e.g., extracting emails, phone numbers, and special string sequences).
- Matching groups: Using parentheses for extracting or pattern grouping.
- Student practicals: Students are tasked with forming patterns to extract titles and emails from a string.

Handling Special Characters

- Demonstrates the importance of escaping characters like `.` for literal matches (e.g., `\.`) in patterns to distinguish from their meta-meaning.
- Shows matching only digits, non-digits, spaces, non-spaces, dots, etc., using character sets or escapes.

Grouping and Alternations

- Using `()` for grouping specific patterns.
- Using `|` for "either/or" pattern alternatives.
- Clarification about grouping differences and nested repetitions.

Assignments and Practice

- Students are assigned to extract titles such as 'Mr.', 'Mrs.' and emails from a sample text using custom regex patterns.

- Assignments are to be emailed; solutions provided after review.
- Emphasis on practicing with real examples to solidify regex use.

Transition to Next Module

- Completion of the core Python module, upcoming is NumPy and Pandas for data science.
- Announcements on future classes and assignments related to NumPy, Pandas, Object-Oriented Programming.

Instructor and Student Interactions

- Ongoing Q&A covering repetition quantifiers, pattern construct nuances, practical implementation doubts.
- Students clarify details about grouping, quantifiers, match vs. search, literal vs. meta-characters.
- Encouragement to refer to Google for further regex resources and wait for assignments and solution sets uploaded by the instructor.

Note:

The summary strictly follows the sequence and details from the lecture transcript. Technical explanations, interactions, and assignment references are included as presented.